Monorepo Migration

Execution Playbook

Paper-Pro-Market | paper-market/core + apps/web + apps/market-engine

5 Phases • Step-by-step commands • Exact file moves • Rollback guides

Overview & Architecture Target

This document is the ground truth for migrating Paper-Pro-Market from a single-root Next.js monolith into a proper Turborepo workspace. It is structured as exact, ordered steps — not abstract goals.

WHY	Root lib/ has no enforced boundary. Next.js, Fastify market-engine, and pure domain logic are all mixed. Market-engine duplicates schema, db, types, and symbol-normalization. One bad import can pull WebSocket or React code into the wrong runtime.

Final Target Structure

```
paper-pro-market/
  package.json          ← workspace root (pnpm workspaces)
  turbo.json            ← Turborepo pipeline
  packages/
    core/               ← @paper-market/core
      src/db/           ← Drizzle schema (single source of truth)
      src/market/       ← types, symbol-normalization, time
      src/trading/      ← pure math (futures-margin, option-margin)
      src/validation/   ← OMS, wallet, options, instruments
      src/instruments/  ← types + pure repo interfaces
      src/analysis/     ← indicator-engine (pure compute)
  apps/
    web/                ← Next.js (current root → moved here)
    market-engine/      ← Fastify WS (already here, unchanged)
```

Boundary Rules (Non-Negotiable)

packages/core	apps/web	apps/market-engine
No React, Next, Fastify, WS	Can import from core	Can import from core
No browser APIs	Keeps auth, chart-registry, market-ws client	Keeps WS server, LTP writer, engine cache
No framework deps in package.json	Keeps services/, stores/, hooks/	Keeps upstox/, server/ directories
Pure TypeScript + drizzle-orm	Runs as Next.js app	Runs as Fastify app

PHASE 0: PRE-FLIGHT AUDIT

Do this BEFORE writing a single line of migration code — takes ~1 hour

Running this audit before anything prevents surprises mid-migration. The outputs directly inform which files go where in Phase 2.

Step 0.1 — Verify market-engine is truly isolated

Confirm that apps/market-engine does not currently import from the root. If it does, those imports must be resolved FIRST before any moves.

```
grep -r "from '../../" apps/market-engine/src/
grep -r "require('../../" apps/market-engine/src/
```

Expected: zero results. If you see any hits, list them and treat them as blockers for Phase 4.

Step 0.2 — Map lib/ files by purity

Run the following to find which lib/ files import Next.js, React, or framework-specific APIs. These CANNOT go into core:

```
grep -rl "from 'next" lib/
grep -rl "from 'react" lib/
grep -rl "next/headers\|next/cache\|next/navigation" lib/
grep -rl "from 'ws'\|WebSocket" lib/
```

Files that appear in any of these results stay in apps/web. All others are candidates for core.

Step 0.3 — Count @/ alias usages

This tells you the import churn scope when the Next.js app moves to apps/web:

```
grep -r "from '@/" app/ components/ services/ stores/ hooks/ jobs/ lib/ --include="*.ts"
--include="*.tsx" | wc -l
```

If this number is over 1000, plan an extra day for the apps/web move. The alias fix is mechanical but time-consuming.

Step 0.4 — List market-engine duplicates

These are the files in market-engine that duplicate root lib/ and will be deleted in Phase 4:

```
apps/market-engine/src/lib/schema.ts   ← duplicates lib/db/schema/
apps/market-engine/src/lib/db.ts       ← duplicates lib/db/index.ts
apps/market-engine/src/core/symbol-normalization.ts  ← duplicates lib/market/symbol-
normalization.ts
apps/market-engine/src/core/types.ts   ← duplicates lib/types/
```

Step 0.5 — Confirm pnpm version

This plan uses pnpm workspaces. Check your current state:

```
pnpm --version          # need 8+
node --version          # need 18+
```

If on npm workspaces: the workspace:* protocol differs slightly, but the concepts are identical. Adjust accordingly.

CHECKPOINT	Save the audit results. You need: (a) list of lib/ files that import Next/React, (b) @/ alias count, (c) confirmation market-engine has zero root imports.

PHASE 1: TURBOREPO SETUP

No code moves yet — just scaffolding. ~2-3 hours

RULE Do NOT move any files in this phase. Only create new config files. Your Next.js app still runs from root at the end of this phase.

Step 1.1 — Convert root to pnpm workspace

Add a pnpm-workspace.yaml at repo root:

```
# pnpm-workspace.yaml
packages:
  - "apps/*"
  - "packages/*"
```

Edit root package.json — add these fields (do NOT remove existing scripts yet):

```
"name": "@paper-market/root",
"private": true,
"workspaces": ["apps/*", "packages/*"]
```

Step 1.2 — Ensure market-engine is a proper workspace member

Check apps/market-engine/package.json. It should have a "name" field:

```
"name": "@paper-market/market-engine"
```

If missing, add it. Then from root run:

```
pnpm install
```

pnpm should now recognize market-engine as a workspace member.

Step 1.3 — Add Turborepo

```
pnpm add -Dw turbo
```

Create turbo.json at root:

```
{
  "$schema": "https://turbo.build/schema.json",
  "tasks": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": [".next/**", "dist/**", "!.next/cache/**"]
    },
    "dev": {
      "cache": false,
      "persistent": true
    },
    "lint": { "dependsOn": ["^build"] },
    "type-check": { "dependsOn": ["^build"] }
  }
}
```

Step 1.4 — Verify nothing is broken

Run the Next.js app from root as before. Turborepo is not yet running anything — just scaffolded:

```
pnpm dev          # should still start Next from root
cd apps/market-engine && pnpm dev   # should still work
```

CHECKPOINT	Git commit: "chore: add turborepo + pnpm workspaces scaffold". The app is fully functional. Nothing moved.

PHASE 2: MOVE NEXT.JS TO apps/web

The riskiest phase. Budget a full day. ~6-8 hours

WARNING This is the phase most teams underestimate. The @/ alias affects hundreds of files. Do this on a dedicated branch with the app running at every checkpoint.

Step 2.1 — Create apps/web directory structure

```
mkdir -p apps/web
```

Step 2.2 — Move these directories into apps/web/

Move each of these from root into apps/web/:

Move from root	To apps/web/
app/	apps/web/app/
components/	apps/web/components/
hooks/	apps/web/hooks/
stores/	apps/web/stores/
services/	apps/web/services/
jobs/	apps/web/jobs/
lib/	apps/web/lib/
content/	apps/web/content/
types/	apps/web/types/
public/	apps/web/public/
middleware.ts	apps/web/middleware.ts
globals.css	apps/web/globals.css

Step 2.3 — Copy (not move) config files into apps/web/

These config files need to live next to the Next app. Copy rather than move — keep originals until verified:

Copy from root	To apps/web/
next.config.js	apps/web/next.config.js
tailwind.config.ts	apps/web/tailwind.config.ts
postcss.config.js	apps/web/postcss.config.js
tsconfig.json	apps/web/tsconfig.json (new — see below)
components.json	apps/web/components.json
drizzle.config.ts	apps/web/drizzle.config.ts

Step 2.4 — Create apps/web/package.json

```
{
  "name": "@paper-market/web",
  "private": true,
```

```
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "type-check": "tsc --noEmit",
    "db:generate": "drizzle-kit generate",
    "db:migrate": "drizzle-kit migrate"
  }
}
```

Then copy the dependencies block from root package.json into this file. Remove any deps that are only used by market-engine (ws, fastify, etc.).

Step 2.5 — Create apps/web/tsconfig.json

This is critical — the @/ alias must now resolve relative to apps/web/:

```
{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": ["./*"]
    },
    "plugins": [{ "name": "next" }]
  },
  "include": ["**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

Create a tsconfig.base.json at repo root with shared compiler options (strict, target, moduleResolution, etc.) so both apps inherit from it.

Step 2.6 — Update next.config.js in apps/web

Add the transpilePackages entry now (will be used in Phase 3):

```
/** @type {import("next").NextConfig} */
const nextConfig = {
  transpilePackages: ["@paper-market/core"],
  // ... rest of your existing config
};
module.exports = nextConfig;
```

Step 2.7 — Install and verify

```
pnpm install
cd apps/web && pnpm dev
```

The Next app should start on its usual port. Fix any import errors — they will typically be:

- Missing env vars: copy .env.local to apps/web/.env.local
- Drizzle config path issues: update DATABASE_URL path in drizzle.config.ts
- next-auth path mismatches: update NEXTAUTH_URL if needed

Step 2.8 — Update root package.json scripts

Replace the root dev/build scripts to delegate to Turborepo:

```
"scripts": {
  "dev": "turbo run dev",
  "build": "turbo run build",
  "web": "pnpm --filter @paper-market/web dev",
  "engine": "pnpm --filter @paper-market/market-engine dev"
}
```

CHECKPOINT	Git commit: "feat: move Next.js app to apps/web". Run full smoke test: login, place a trade, check positions, check websocket feeds.

PHASE 3: CREATE packages/core

Low risk — new package, no moves yet. ~3-4 hours

APPROACH Start by moving ONE small module end-to-end to validate the full toolchain (workspace resolution, TypeScript paths, Turbo build order). Only then move the rest.

Step 3.1 — Scaffold the core package

```
mkdir -p packages/core/src/{db/schema,market,trading,validation,instruments,analysis}
touch packages/core/src/index.ts
```

Step 3.2 — Create packages/core/package.json

```
{
  "name": "@paper-market/core",
  "version": "0.0.1",
  "private": true,
  "main": "./src/index.ts",
  "types": "./src/index.ts",
  "exports": {
    ".": "./src/index.ts"
  },
  "scripts": {
    "type-check": "tsc --noEmit",
    "lint": "eslint src/"
  },
  "dependencies": {
    "drizzle-orm": "workspace:*"
  },
  "devDependencies": {
    "typescript": "workspace:*"
  }
}
```

Important: drizzle-orm is allowed. React, Next, Fastify, ws are NOT allowed in this package.json.

Step 3.3 — Create packages/core/tsconfig.json

```
{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": {
    "outDir": "dist",
    "rootDir": "src",
    "declaration": true,
    "declarationMap": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

Step 3.4 — First slice: symbol-normalization

This is your canary. Move ONLY this file first:

- Copy apps/web/lib/market/symbol-normalization.ts to packages/core/src/market/symbol-normalization.ts
- Check its imports — it should have zero framework dependencies
- Export it from packages/core/src/index.ts:

```
export * from "./market/symbol-normalization";
```

In apps/web, add the core dependency:

```
# in apps/web/package.json dependencies:
"@paper-market/core": "workspace:*"

pnpm install
```

Replace ONE import in apps/web to use core:

```
// Before:
import { normalizeSymbol } from "@/lib/market/symbol-normalization";
// After:
import { normalizeSymbol } from "@paper-market/core";
```

Run build and check it works end-to-end. This validates: pnpm workspace resolution, TypeScript paths, Turbo build order. Fix any issues before proceeding.

CHECKPOINT Git commit: "feat: add packages/core with symbol-normalization slice". Full build passes.

PHASE 4: MOVE SHARED DOMAIN INTO CORE

The main migration. Do one group at a time. ~2-3 days

RULE Move ONE group, update imports, run the build, commit. Then move the next group. Never move two groups in the same commit.

Step 4.1 — Group 1: DB Schema (Highest Value, Do First)

Move the Drizzle schema — this is the most impactful change since both apps need it.

Files to move from apps/web/lib/db/schema/ to packages/core/src/db/schema/:

- index.ts
- oms.schema.ts
- users.schema.ts
- wallet.schema.ts
- market.schema.ts
- ledger.schema.ts
- watchlist.schema.ts
- integrations.schema.ts
- write_ahead_journal.schema.ts

Also move apps/web/lib/db/index.ts (the drizzle client factory) to packages/core/src/db/index.ts. IMPORTANT: The db client must accept a connection string as a parameter — do NOT hardcode process.env here:

```
// packages/core/src/db/index.ts
import { drizzle } from "drizzle-orm/postgres-js";
import postgres from "postgres";
import * as schema from "./schema";

export function createDb(connectionString: string) {
  const client = postgres(connectionString);
  return drizzle(client, { schema });
}
export * from "./schema";
```

In apps/web, update the db instantiation to call createDb(process.env.DATABASE_URL). The env var stays in apps/web.

Leave drizzle/ migrations folder at root (or move to packages/core). Convention: keep them in packages/core/drizzle/ and update drizzle.config.ts in apps/web to point there:

```
// apps/web/drizzle.config.ts
export default {
  schema: "../../packages/core/src/db/schema/index.ts",
  out: "../../packages/core/drizzle",
  // ...
}
```

CHECKPOINT	Commit: "feat(core): move db schema to packages/core". Build and run full app. Check all API routes that query the DB.

Step 4.2 — Group 2: Types

Move these files to packages/core/src/:

From (apps/web)	To (packages/core/src)
`lib/types/instrument.types.ts`	instruments/instrument.types.ts
`lib/market-data/types.ts`	market/market-data.types.ts
`types/order.types.ts`	trading/order.types.ts
`types/position.types.ts`	trading/position.types.ts
`types/pnl.types.ts`	trading/pnl.types.ts
`types/general.types.ts`	types/general.types.ts

Note: types/dashboard.types.ts, types/user.types.ts, types/equity.types.ts — check if they import React or Next before moving. If they do, they stay in apps/web.

CHECKPOINT	Commit: "feat(core): move shared types to packages/core".

Step 4.3 — Group 3: Validation

All files in apps/web/lib/validation/ should be framework-free. Move to packages/core/src/validation/:

- auth.ts
- instruments.ts
- oms.ts
- option-chain.ts
- options-strategy.ts
- wallet.ts

CHECKPOINT	Commit: "feat(core): move validation to packages/core".

Step 4.4 — Group 4: Pure Market & Trading Math

Move from apps/web/lib/market/ and apps/web/lib/trading/ — ONLY pure compute files:

File	Action
`lib/market/time.ts`	MOVE → core/src/market/time.ts
`lib/market/symbol-normalization.ts`	ALREADY IN CORE (Phase 3)
`lib/market/upstox-quote-normalization.ts`	MOVE → core/src/market/
`lib/trading/futures-margin.ts`	MOVE → core/src/trading/
`lib/trading/option-margin.ts`	MOVE → core/src/trading/
`lib/fno-utils.ts`	MOVE → core/src/trading/
`lib/fno-payoff-utils.ts`	MOVE → core/src/trading/
`lib/expiry-utils.ts`	MOVE → core/src/market/
`lib/market-hours.ts`	MOVE → core/src/market/

File	Action
lib/trading/candle-engine.ts	CHECK FIRST — see note
lib/trading/chart-registry.ts	STAY in apps/web
lib/trading/chart-controller.ts	STAY in apps/web
lib/trading/init-realtime.ts	STAY in apps/web
lib/market/market-cache.ts	STAY in apps/web (uses Redis)
lib/market/candle-orchestrator.ts	STAY in apps/web

Note on candle-engine.ts: Check if it imports any browser APIs, WebSocket, or React. If it is pure compute (just candlestick math), move to core. If it holds state or references DOM/WS, keep in apps/web.

CHECKPOINT Commit: "feat(core): move pure market and trading math to packages/core".

Step 4.5 — Group 5: Instruments and Analysis

Move pure instrument and analysis code:

- lib/instruments/repository.ts (interfaces only) → core/src/instruments/repository.ts
- lib/analysis/indicator-engine.ts → core/src/analysis/indicator-engine.ts (CHECK for browser deps first)

DO NOT move:

- lib/instruments/instrument-sync.service.ts — stays in apps/web (calls Upstox API)
- lib/instruments/index.ts — re-export from apps/web pointing to core

CHECKPOINT Commit: "feat(core): move instruments and analysis to packages/core". Full integration test.

Step 4.6 — Export everything from core/src/index.ts

The public API of core should be one clean barrel export:

```
// packages/core/src/index.ts
export * from "./db";
export * from "./market/symbol-normalization";
export * from "./market/time";
export * from "./market/market-data.types";
export * from "./market/expiry-utils";
export * from "./market/market-hours";
export * from "./trading/futures-margin";
export * from "./trading/option-margin";
export * from "./trading/fno-utils";
export * from "./validation";
export * from "./instruments";
export * from "./analysis";
```

PHASE 5: MARKET-ENGINE ADOPTS CORE

Delete duplicates. ~3-4 hours

DEPENDENCY Complete Phase 4 fully and verify the build before starting Phase 5. Market-engine is a production service; treat changes carefully.

Step 5.1 — Add core dependency to market-engine

In apps/market-engine/package.json:

```
"dependencies": {
  "@paper-market/core": "workspace:*",
  // ... existing deps
}
pnpm install
```

Step 5.2 — Replace duplicates one at a time

market-engine file (DELETE)	Replace with
src/lib/schema.ts	import from @paper-market/core
src/core/symbol-normalization.ts	import from @paper-market/core
src/core/types.ts	import from @paper-market/core
src/lib/db.ts	import createDb from @paper-market/core

KEEP in market-engine (these are engine-specific, do NOT put in core):

- src/lib/redis.ts
- src/lib/logger.ts
- src/lib/ltp-cache-writer.ts
- src/lib/market-cache.ts
- src/core/candle-engine.ts (engine version — distinct from web version)
- src/core/tick-bus.ts
- src/server/ws-server.ts
- src/upstox/ (all files)

CHECKPOINT Commit: "feat(market-engine): use @paper-market/core, remove duplicates". Start market-engine and verify DB connection and Upstox feed.

PHASE 6: ENFORCE BOUNDARIES

~2 hours — prevents future bleed

Step 6.1 — ESLint import restrictions

Add to apps/market-engine/.eslintrc (or eslint.config.js):

```
"no-restricted-imports": ["error", {
  "patterns": [
    { "group": ["@paper-market/web", "apps/web/*"],
      "message": "market-engine must not import from apps/web" },
    { "group": ["next/*", "react"],
      "message": "market-engine must not import Next/React" }
  ]
}]
```

Add to packages/core/.eslintrc:

```
"no-restricted-imports": ["error", {
  "patterns": [
    { "group": ["next/*", "react", "fastify", "ws"],
      "message": "core must have no framework dependencies" },
    { "group": ["apps/web/*", "apps/market-engine/*"],
      "message": "core must not import from apps" }
  ]
}]
```

Step 6.2 — CI build check

Add to your CI pipeline (GitHub Actions or Railway):

```
# .github/workflows/ci.yml
- name: Check core has no framework deps
  run: |
    if grep -r ""next"\|"react"\|"fastify"\|"ws"" packages/core/package.json; then
      echo "FAIL: core has framework dependency" && exit 1
    fi
- name: Turbo build all
  run: pnpm turbo run build
```

Step 6.3 — Add README to packages/core

Create packages/core/README.md with these rules so future contributors know:

- What belongs in core: types, validation, pure math, db schema
- What does NOT belong: auth, chart UI, WS client, services, stores
- How to add to core: add to src/, export from index.ts, no framework imports

CHECKPOINT Final commit: "chore: enforce workspace boundaries with ESLint and CI". Migration complete.

Rollback Guide

Each phase is designed to be independently revertible. Use git branches.

Phase	Rollback action	Risk if abandoned mid-phase
Phase 1 (Turbo)	Delete pnpm-workspace.yaml + turbo.json, revert package.json	Zero — only config files added
Phase 2 (apps/web)	git revert the move commit, restore root structure	High — @/ aliases broken until resolved
Phase 3 (core scaffold)	Delete packages/core, revert imports	Low — one file changed
Phase 4 (domain moves)	Per-group: git revert individual group commit	Medium — import paths changed
Phase 5 (engine)	Restore deleted engine files from git, remove core dep	Low — market-engine is separate process

Time Estimates & Sequencing

Phase	Min	Max	Biggest risk
Phase 0: Audit	1h	2h	@/ alias count surprise
Phase 1: Turborepo	2h	3h	pnpm vs npm workspace syntax
Phase 2: apps/web move	6h	10h	env vars + @/ alias churn
Phase 3: core scaffold	2h	4h	Turbo build order config
Phase 4: domain moves	8h	16h	Hidden framework imports
Phase 5: engine adopts core	3h	5h	DB connection string handling
Phase 6: boundaries	1h	2h	ESLint config syntax
Total	**23h**	**42h**	

RECOMMENDATION Spread this across 2 weeks of normal work: Phase 0+1 in week 1 day 1, Phase 2 in a dedicated day, Phases 3-4 across 3-4 days, Phases 5-6 in the last day. Never do two phases in the same day.